

СОВРЕМЕННЫЕ ТЕХНОЛОГИИ РАЗРАБОТКИ WEB- ПРИЛОЖЕНИЙ

ЛЕКЦИЯ 11с

Темы занятия

Асинхронное выполнение и Promise

Коллекции Map, Set, WeakMap и WeakSet

Буферные и типизированные массивы

Асинхронное выполнение

Асинхронный код – код, который выполняется в параллельном вычислительном потоке.

Когда стоит использовать:

- работа с сетью
- работа с диском
- фоновые вычисления
- анимация интерфейса
- . . .

Функция `setTimeout()`

Глобальный объект в браузерах и Node.js имеет функцию `setTimeout()`, позволяющую задавать вызов некой функции через заданный период времени.

Аргументы `setTimeout()`:

- Функция для исполнения `f`
- Период времени в миллисекундах
- Аргументы функции `f` (не поддерживается в <IE9)

Функция setTimeout() – пример

```
function func(str) {  
    alert(str);  
}
```

```
alert("Start");  
setTimeout(func, 5000, "Hello");  
alert("Stop");
```

```
// setTimeout() возвращает идентификатор таймера  
let id = setTimeout(func, 2000, "Cancel");
```

```
// функция clearTimeout() останавливает таймер  
clearTimeout(id);  
alert("Clear");
```

Объект XMLHttpRequest

Объект XMLHttpRequest даёт возможность делать асинхронные HTTP-запросы к серверу.

Несмотря на слово «XML» в названии, XMLHttpRequest может работать с любыми данными, а не только с XML.

Некоторые элементы XMLHttpRequest

Элемент	Описание
<code>open()</code>	Настройка объекта – указывается HTTP-глагол, URI, и true/false (асинхронное или синхронное использование)
<code>send()</code>	Отправка запроса. Аргумент – тело HTTP-запроса
<code>abort()</code>	Прерывает выполнение запроса
<code>status</code>	HTTP-код ответа. Может быть равен 0, если сервер не ответил или при запросе на другой домен
<code>statusText</code>	Текстовое описание HTTP-кода: OK, Not Found, Forbidden
<code>responseText</code>	Текст ответа сервера
<code>onreadystatechange</code>	Функция – обработчик события изменения <code>status</code>
<code>load</code>	Событие «Запрос был успешно (без ошибок) завершён»
<code>error</code>	Событие «Произошла ошибка»

XMLHttpRequest – пример работы

```
let req = new XMLHttpRequest();

// важно - кросс-доменные запросы не работают
req.open("GET", "/mypage.aspx", true);
req.onreadystatechange = function (aEvt) {
    if (req.readyState == 4) {
        if (req.status == 200)
            alert(req.responseText.length);
        else
            alert(req.status);
    }
};

req.send(null);
```


Promise

Promise («промис») – объект, используемый для выполнения отложенных и асинхронных операций.

Промис хранит своё состояние. Вначале это `pending` (*ожидание*), затем – `fulfilled` (*выполнено успешно*) или `rejected` (*выполнено с ошибкой*).

Для промиса можно задать функции-обработчики:

- `onFulfilled`: вызывается при переходе в состояние `fulfilled`
- `onRejected`: вызывается при переходе в `rejected`

Promise — схема использования

1. Код, которому надо сделать что-то асинхронно, создаёт объект-промис и возвращает его.
2. Внешний код, получив промис, навешивает на него обработчики.
3. По завершении процесса асинхронный код из пункта 1 переводит промис в состояние `fulfilled` (с результатом) или `rejected` (с ошибкой). При этом во внешнем коде автоматически вызываются соответствующие обработчики.

Promise — схема использования

Синтаксис создания промиса:

```
let promise = new Promise(function(resolve, reject) {  
    // Эта функция будет вызвана автоматически  
    // В ней можно делать любые асинхронные операции,  
    // А когда они завершатся — нужно вызвать одно из:  
    // resolve(результат) при успешном выполнении  
    // reject(ошибка) при ошибке  
})
```

Универсальный метод для навешивания обработчиков:

```
promise.then(onFulfilled, onRejected)
```

Promise – пример (а)

```
// создадим промис
// асинхронная операция:
// через 1 секунд срабатывает функция
let pr = new Promise((resolve, reject) => {
  setTimeout(() => {
    // переведёт промис в состояние fulfilled
    // с результатом "result"
    resolve("result");
  }, 1000);
});
```

Promise – пример (b)

```
// promise.then() навешивает обработчики
// на успешный результат или ошибку
promise.then(
  result => {
    // первая функция запустится при вызове resolve
    // result - аргумент resolve
    alert("Fulfilled: " + result);
  },
  error => {
    // вторая функция запустится при вызове reject
    // error - аргумент reject
    alert("Rejected: " + error);
  }
);
```

Promise — нюансы

1. С помощью `then()` можно назначить оба обработчика или только один: `promise.then(onFulfilled)`
2. Чтобы поставить обработчик только на ошибку, вместо `then(null, onRejected)` можно писать `catch(onRejected)`
3. Если в функции промиса происходит синхронный `throw` (или иная ошибка), то вызывается `reject`.
4. Когда промис переходит в состояние «выполнен» — с результатом или ошибкой — это навсегда. То есть, после вызова `resolve/reject` промис уже не может «передумать».

Промисификация

Промисификация – создание «обёртки» для асинхронного функционала, возвращающей промис.

В качестве примера сделаем такую обёртку для запросов при помощи XMLHttpRequest.

Функция `httpGet(url)` будет возвращать промис, который при успешной загрузке данных с `url` будет переходить в `fulfilled` с этими данными, а при ошибке – в `rejected` с информацией об ошибке.

Промисификация – пример

```
function httpGet(url) {  
    return new Promise(function (resolve, reject) {  
        var xhr = new XMLHttpRequest();  
        xhr.open('GET', url, true);  
        xhr.onload = function () {  
            if (this.status == 200) {  
                resolve(this.response);  
            } else {  
                var error = new Error(this.statusText);  
                error.code = this.status;  
                reject(error);  
            }  
        };  
        xhr.onerror = function () {  
            reject(new Error("Network Error"));  
        };  
        xhr.send();  
    });  
}
```


Цепочки промисов

Метод `then()` допускает цепочечный вызов:

```
promise(...)  
  .then(...)  
  .then(...)  
  .then(...)
```

При этом, если `then()` вернул промис, то по цепочке будет передан не сам этот промис, а его результат.

Для обработки ошибок следует вместо `then()` в цепочке использовать `catch()`.

Цепочки промисов — пример

```
// сделать запрос
httpGet('/article/promise/user.json')
  // получить данные о пользователе, передать дальше
  .then(response => {
    let user = JSON.parse(response);
    return user;
  })
  // получить информацию с github
  .then(user => {
    return httpGet(`https://api.github.com/users/${user.name}`);
  })
  // вывести аватар на 3 секунды (можно с анимацией)
  .then(githubUser => {
    githubUser = JSON.parse(githubUser);
    let img = new Image();
    img.src = githubUser.avatar_url;
    img.className = "promise-avatar-example";
    document.body.appendChild(img);
    setTimeout(() => img.remove(), 3000);
  });
```

Статические методы класса Promise()

`Promise.all(iterable)` получает итерируемый объект промисов и возвращает промис, который ждёт, пока все переданные промисы завершатся, и переходит в состояние «выполнено» с массивом их результатов.

`Promise.race(iterable)` получает итерируемый объект промисов. Результатом будет только первый успешно выполнившийся промис из списка, остальные игнорируются.

Промисы и генераторы

Комбинация промисов и генераторов позволяет писать «плоский» асинхронный код.

Общий принцип:

- В генераторе `yield` возвращает промисы.
- Специальная функция `execute(generator)` запускает генератор, и вызовами `next()` получает из него промисы. А когда очередной промис выполнится, возвращает его результат в генератор следующим `next()`.
- Последнее значение генератора (`done: true`) `execute()` уже обрабатывает как окончательный результат.

Промисы и генераторы – пример (а)

```
// генератор для получения и показа аватара
function* showUserAvatar() {
  let userFetch = yield fetch('/article/generator/user.json');
  let userInfo = yield userFetch.json();
  let githubFetch =
    yield fetch(`https://api.github.com/users/${userInfo.name}`);
  let githubUserInfo = yield githubFetch.json();
  let img = new Image();
  img.src = githubUserInfo.avatar_url;
  img.className = "promise-avatar-example";
  document.body.appendChild(img);
  yield new Promise(resolve => setTimeout(resolve, 3000));
  img.remove();
  return img.src;
}
```

Промисы и генераторы – пример (b)

```
// вспомогательная функция-чернорабочий для выполнения промисов из generator
function execute(generator, yieldValue) {
  let next = generator.next(yieldValue);
  if (!next.done) {
    next.value.then(
      result => execute(generator, result),
      err => generator.throw(err)
    );
  } else {
    // обработаем результат return из генератора
    // обычно здесь вызов callback или что-то в этом духе
    alert(next.value);
  }
}

execute(showUserAvatar());
```

Коллекция Map

Map – это коллекция для хранения пар «*ключ-значение*». Ключ может быть абсолютно любым. Ключи сохраняются как есть, без преобразования типов (напомним, что в объекте ключами могут быть только строки).

Проверка ключей на равенство почти аналогична оператору `===`, но NaN считается равным NaN.

Свойства и методы Map

size	Количество пар в экземпляре Map
clear()	Удаляет все пары из экземпляра Map
delete()	Пытается удалить пару по ключу. Возвращает true , если пара была в Map
entries()	Итератор из массивов <i>[ключ, значение]</i> в порядке вставки пар
forEach()	Выполняет аргумент-функцию для каждой пары
get()	Получает элемент по ключу (нет ключа – возвращает undefined)
has()	Проверяет, есть ли ключ в экземпляре Map
keys()	Итератор ключей в порядке вставки пар
set()	Устанавливает значение для ключа и возвращает экземпляр Map
values()	Итератор значений в порядке вставки пар
<i>Итератор</i>	Итератор из массивов <i>[ключ, значение]</i> в порядке вставки пар

Коллекция Map – пример 1

```
let map = new Map();
map.set("1", "str");    // ключ - строка
map.set(1, "num");      // ключ - число
map.set(true, "bool");  // ключ - булево значение

alert(map.size);        // 3 – количество пар

// в обычном объекте это было бы одно и то же
alert(map.get(1));      // num
alert(map.get("1"));    // str

map.delete("1");
alert(map.has("1") ? "Yes" : "No");    // No
```

Коллекция Map – пример 2

```
// аргументом конструктора может быть итерируемый набор пар
let map = new Map([["1", "str"], [1, "num"]]);

// NaN как ключ
map.set(NaN, "Not a Number");

// цепочка set()
map.set(1, "one").set(2, "two");

// перебор пар
for(let [key, value] of map) alert(key + " = " + value);

// перебор пар (альтернатива)
for(let [key, value] of map.entries()) alert(key + " = " + value);

// перебор ключей
for (let key of map.keys()) alert(key);

// перебор значений
for (let value of map.values()) alert(value);
map.forEach((value, key, map) => alert(key + " = " + value));
```

Коллекция Set

Set – множество неповторяющихся значений.

Коллекция Set имеет схожесть с Map: ключом может быть абсолютно произвольное значение, которое сохраняется без преобразования типов.

Проверка ключей на равенство в Set осуществляется аналогично проверке в Map (NaN считается равно NaN).

Свойства и методы Set

<code>size</code>	Количество значений в экземпляре Set
<code>add()</code>	Добавляет значение в Set (если его там нет), возвращает экземпляр Set
<code>clear()</code>	Удаляет все значения из экземпляра Set
<code>delete()</code>	Удаляет указанное значение. Возвращает <code>true</code> , если было что удалять
<code>entries()</code>	Итератор из массивов [<i>значение</i> , <i>значение</i>] в порядке вставки значений
<code>forEach()</code>	Выполняет аргумент-функцию для каждого значения
<code>has()</code>	Проверяет, есть ли значение в экземпляре Set
<code>keys()</code>	Итератор значений в порядке вставки (то же, что и <code>values()</code>)
<code>values()</code>	Итератор значений в порядке вставки
<i>Итератор</i>	Итератор значений в порядке вставки

Коллекция Set – пример

// конструктору Set можно передать итерируемый объект

```
let set = new Set([1, "2", false]);
```

```
set.add(NaN).add(1);
```

// 1, 2, false, NaN

```
set.forEach((value, key, set) => alert(key));
```

```
set.delete(false);
```

// 1, 2, NaN

```
for (let value of set.values()) alert(value);
```

Коллекции WeakMap и WeakSet

WeakSet – особый Set, не препятствующий сборщику мусора удалять свои элементы. То есть, если некий объект присутствует **только** в WeakSet, он удаляется из памяти.

Аналогично ведёт себя WeakMap для Map.

Важно: ключи в WeakMap и значения в WeakSet – только объекты (иначе TypeError).

Применение WeakSet/WeakMap

У нас есть *элементы* на странице. Мы для элементов хотим хранить *обработчики* событий.

Поместим пару *элемент-обработчик* в WeakMap (элемент будет ключом).

Если удалить элемент на странице, сборщик мусора автоматически удалит и обработчик для элемента из WeakMap. Таким образом, WeakMap избавляет нас от необходимости вручную удалять вспомогательные данные, когда удалён основной объект.

Ограничения WeakSet/WeakMap

- Нет свойства `size`
- Нельзя перебрать элементы (ключи, значения)
- Нет метода `forEach()`
- Нет метода `clear()`

Ограничения связаны с тем, что с `Weak`-коллекциями в **независимом потоке** работает сборщик мусора.

Буферные и типизированные массивы

Классические массивы ECMAScript обладают рядом особенностей: динамическое изменение размера, хранение элементов разных типов.

Эти особенности порождают очевидные проблемы: низкая скорость работы и большое потребление памяти.

Для решения этих проблем ECMAScript 2015 предлагает *буферные массивы и типизированные массивы*.

Буферные массивы

Буферный массив – коллекция байт в памяти. Каждый байт рассматривается как элемент буферного массива.

Размер буферного массива определяется при его создании и не может изменяться динамически.

В момент создания буферного массива все его элементы инициализируются значением 0.

Буферные массивы — создание

Буферный массив создаётся конструктором `ArrayBuffer` с указанием неотрицательного размера в байтах:

```
let buffer = new ArrayBuffer(80);
```

Однако напрямую работать с буферным массивом нельзя. Для обслуживания массива используются либо объекты `DataView`, либо типизированные массивы.

Объект DataView

Объект DataView создаётся на основе существующего буферного массива. Опционально можно указать смещение в буфере и захватываемую длину буфера.

```
let dv = new DataView(buffer);
```

Объект DataView предоставляет методы чтения и записи элементов целочисленных типов:

```
// записали -10 со смещением 0 байт  
dv.setInt32(0, -10, false);  
alert(dv.getInt32(0, false));
```

DataView – методы чтения и записи

Любой метод записи DataView получает три аргумента:

- смещение в буфере
- записываемое число
- (*) **true** (порядок байт little-endian) или **false** (big-endian)

Метод чтения получает два аргумента:

- смещение в буфере
- (*) **true** (порядок байт little-endian) или **false** (big-endian)

DataView – методы чтения и записи

Типы, для которых существуют методы чтения и записи:

Тип	Размер, байт	Описание
Int8	1	8-битовое знаковое целое
UInt8	1	8-битовое беззнаковое целое
Int16	2	16-битовое знаковое целое
UInt16	2	16-битовое беззнаковое целое
Int32	4	32-битовое знаковое целое
UInt32	4	32-битовое беззнаковое целое
Float32	4	32-битовое с плавающей запятой
Float64	8	64-битовое с плавающей запятой

Типизированные массивы

Типизированные массивы позволяют работать с буферными массивами как с обычными массивами.

Для каждого типа из таблицы на предыдущем слайде существует свой массив. К примеру, массив `Int32Array`.

Типизированные массивы поддерживают многие методы обычных массивов `Array`. Для них доступна возможность итерирования.

Пример работы с массивом Int32Array

```
let buffer = new ArrayBuffer(80);

// разные конструкторы
let arr1 = new Int32Array(10);
let arr2 = new Int32Array(arr1);
let arr3 = new Int32Array(buffer); // + смещение и длина

arr1[0] = 10;
arr1[1] = 20;

// выведет все десять элементов (10, 20, остальные 0)
for(let x of arr1) alert(x);
```


Типизированные массивы

Особо упомянем массив `Uint8ClampedArray`. Он весьма похож на `Uint8Array`. Однако эти массивы ведут себя по-разному, если элементу присваивается значение за пределами диапазона `0..255`:

- `Uint8Array` – используется восемь младших байт значения
- `Uint8ClampedArray` – используется `0` (если значение меньше нуля) или `255` (если значение больше, чем `255`)